

Focus On Deep Convolutional Networks

Popular Types of DNNs

- **Fully-Connected NN**
 - feed forward, a.k.a. multilayer perceptron (MLP)
- **Convolutional NN (CNN)**
 - feed forward, sparsely-connected w/ weight sharing
- **Recurrent NN (RNN)**
 - feedback
- **Long Short-Term Memory (LSTM)**
 - feedback + storage

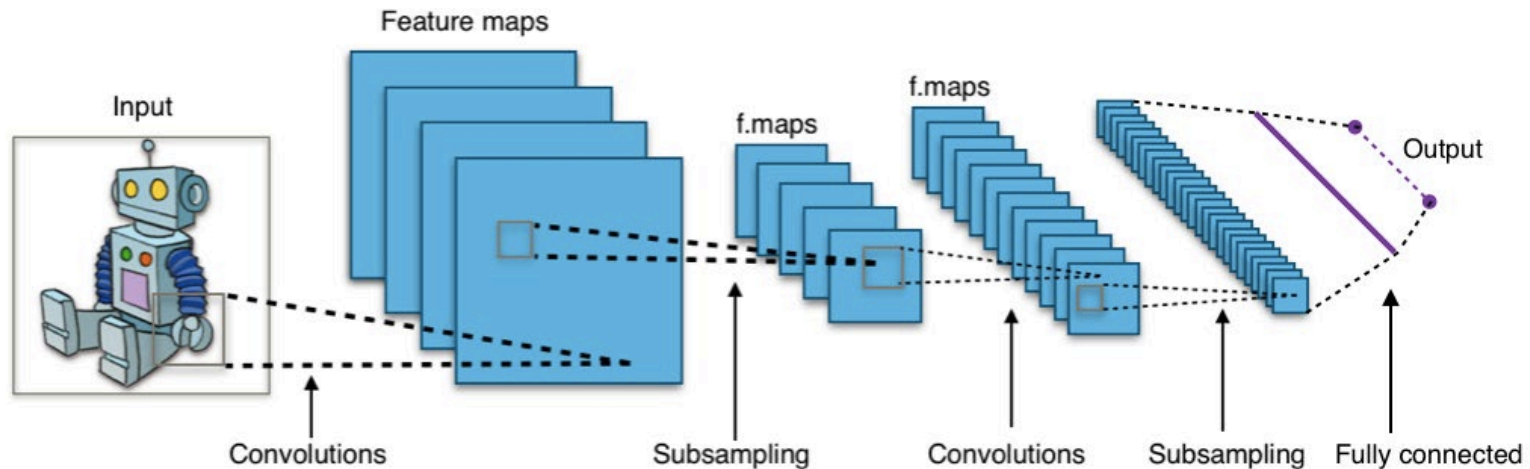
Limitations of MLPs for Vision

- **Not scalable**
 - High number of weights per neuron required
 - ..and therefore high computational cost
- **Ignore “Locality” in data with grid-topology**
 - Input pixels far apart treated equally as pixels close together
- **Prone to overfitting**
 - Due to “full-connectivity” structure

How to improve?

Convolutional Neural Networks

- A convolutional neural network (CNN) is a regularized type of feed-forward network that learns the features by itself via filters (or kernels) optimization.
- The de-facto standard models in DL for vision
 - Now being replaced by Transformers



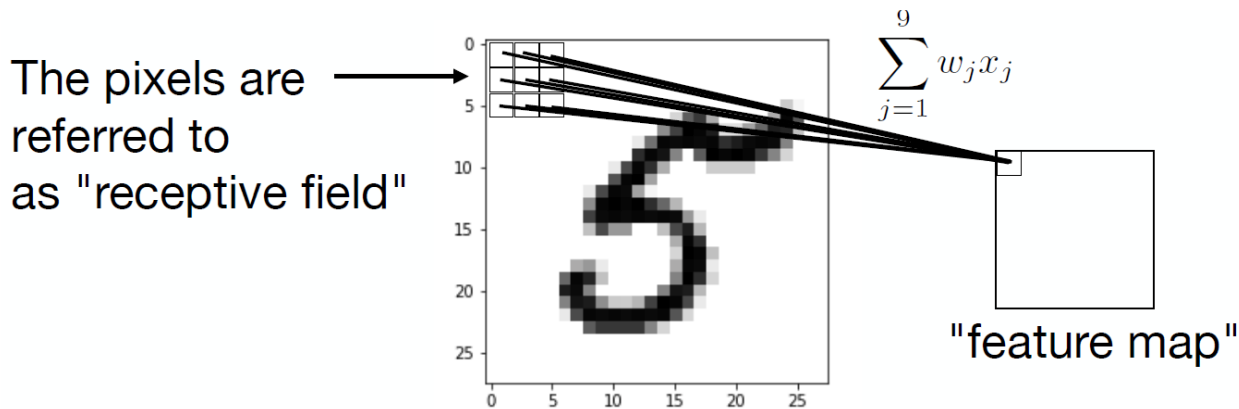
Main Concepts Behind Convolutional Neural Networks

Why CNNs?

- **Sparse-connectivity:** A single element in the feature map is connected to only a small patch of pixels. (This is very different from connecting to the whole input image, in the case of multi-layer perceptrons.)
- **Parameter-sharing:** The same weights are used for different patches of the input image.
- **Many layers:** Combining extracted local patterns to global patterns → **DEEP** Convolutional Neural Networks

Receptive field and Weight Sharing

A "feature detector" (filter, kernel) slides over the inputs to generate a feature map

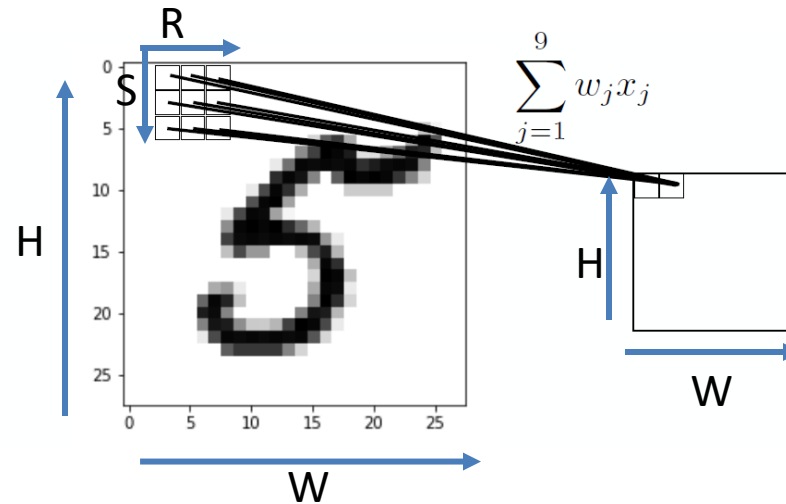


Rationale: A feature detector that works well in one region may also work well in another region

Plus, it is a nice reduction in parameters to fit

Receptive field and Weight Sharing

A "feature detector" (kernel) slides over the inputs to generate a feature map



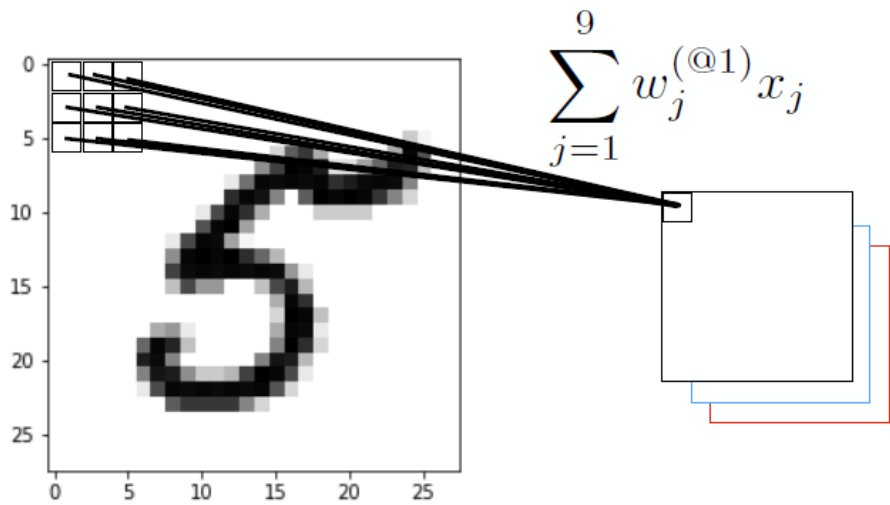
Big reduction in number of trainable weights:

Traditional Neurons: $W \times H$ (neurons) $\times$ $W \times H$ (weights per neuron) $\rightarrow N^2$ ($N=W \times H$)

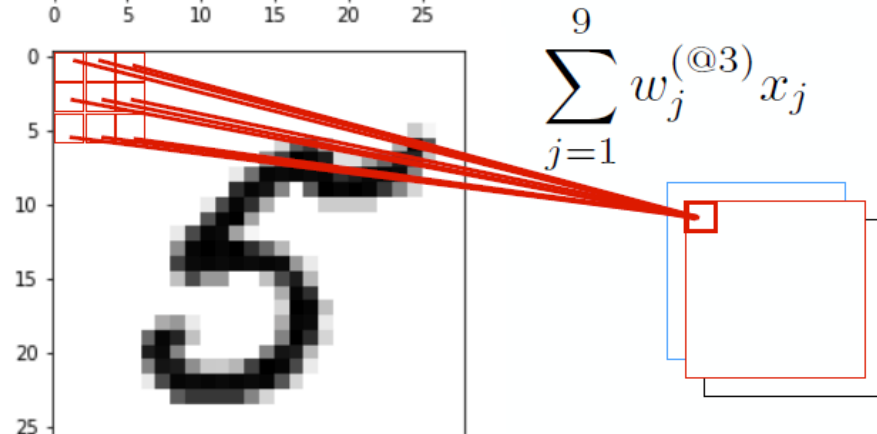
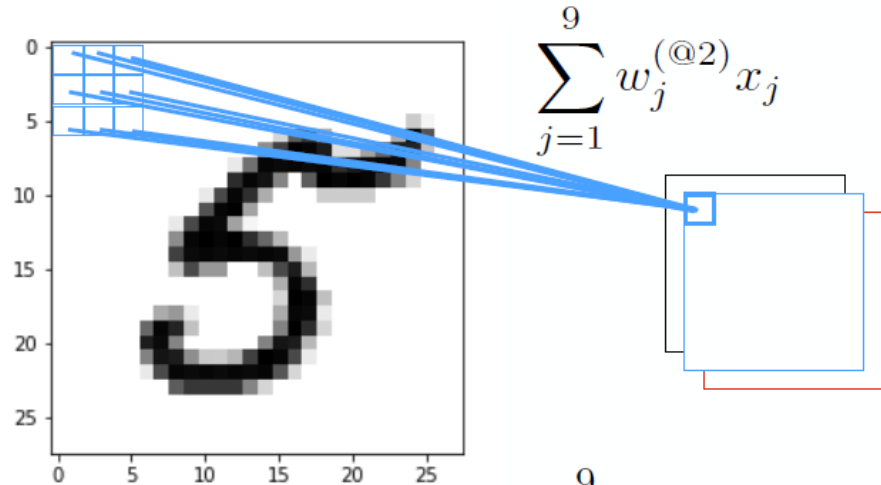
CNN: $S \times R$ (S and R determine the kernel size)

e.g. $C=3$, $W, H=10$

NN $\rightarrow (10 \times 10)^2 = 10.000$ vs. CNN $\rightarrow 3 \times 3 = 9$ weights!



Multiple "feature detectors" (kernels) are used to create multiple feature maps

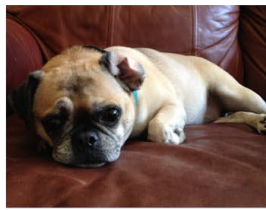


This procedure is called "Convolution" operation and the set of multiple "feature detectors" is called convolution layer or convolution kernel

Preview

[Zeiler and Fergus 2013]

Visualization of VGG-16 by Lane McIntosh. VGG-16 architecture from [Simonyan and Zisserman 2014].

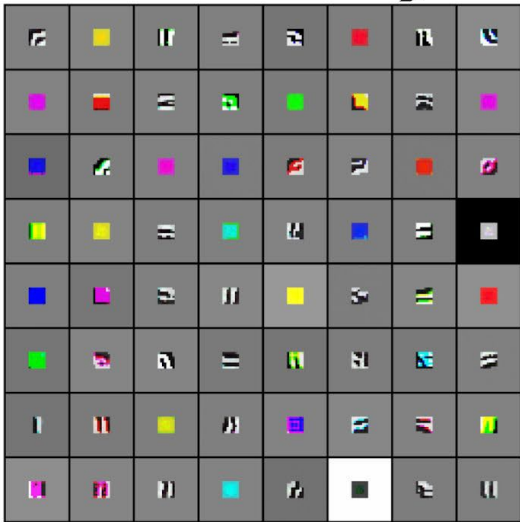


Low-level features

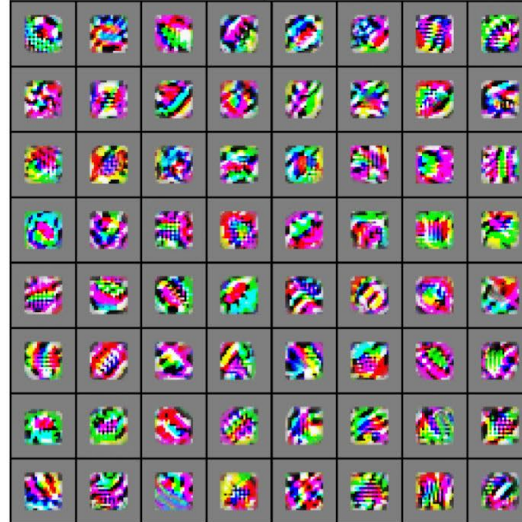
Mid-level features

High-level features

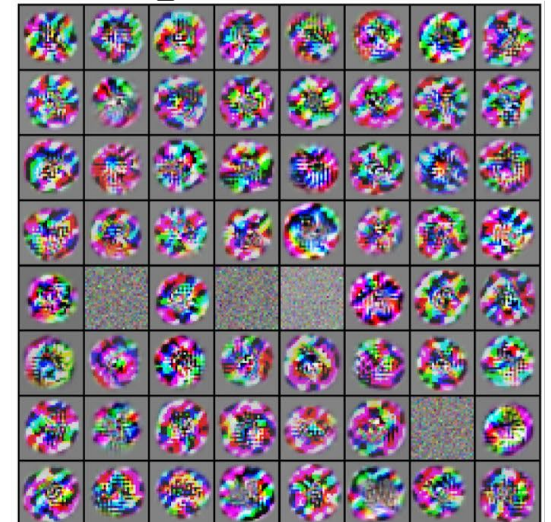
Linearly separable classifier



VGG-16 Conv1_1



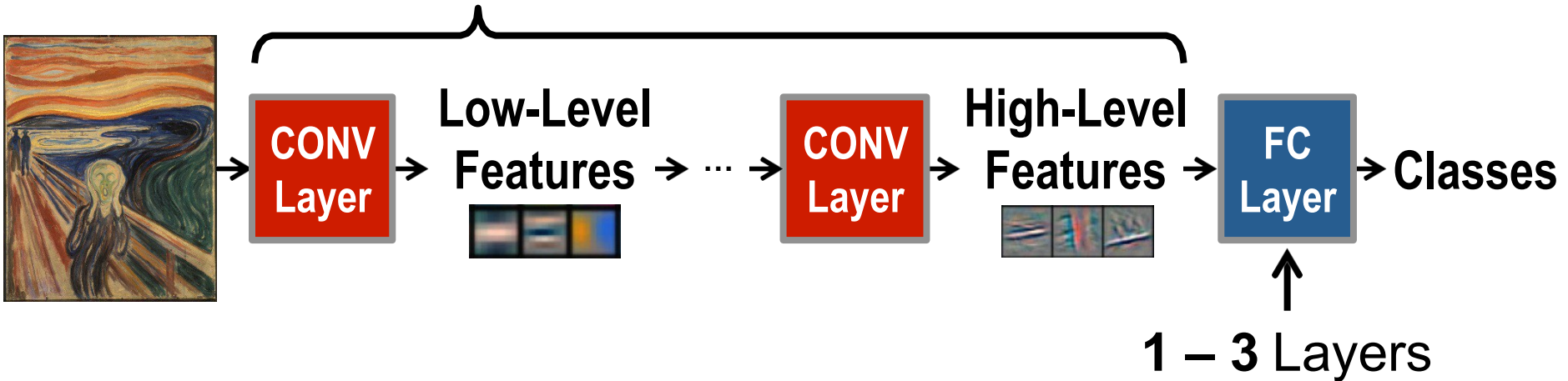
VGG-16 Conv3_2



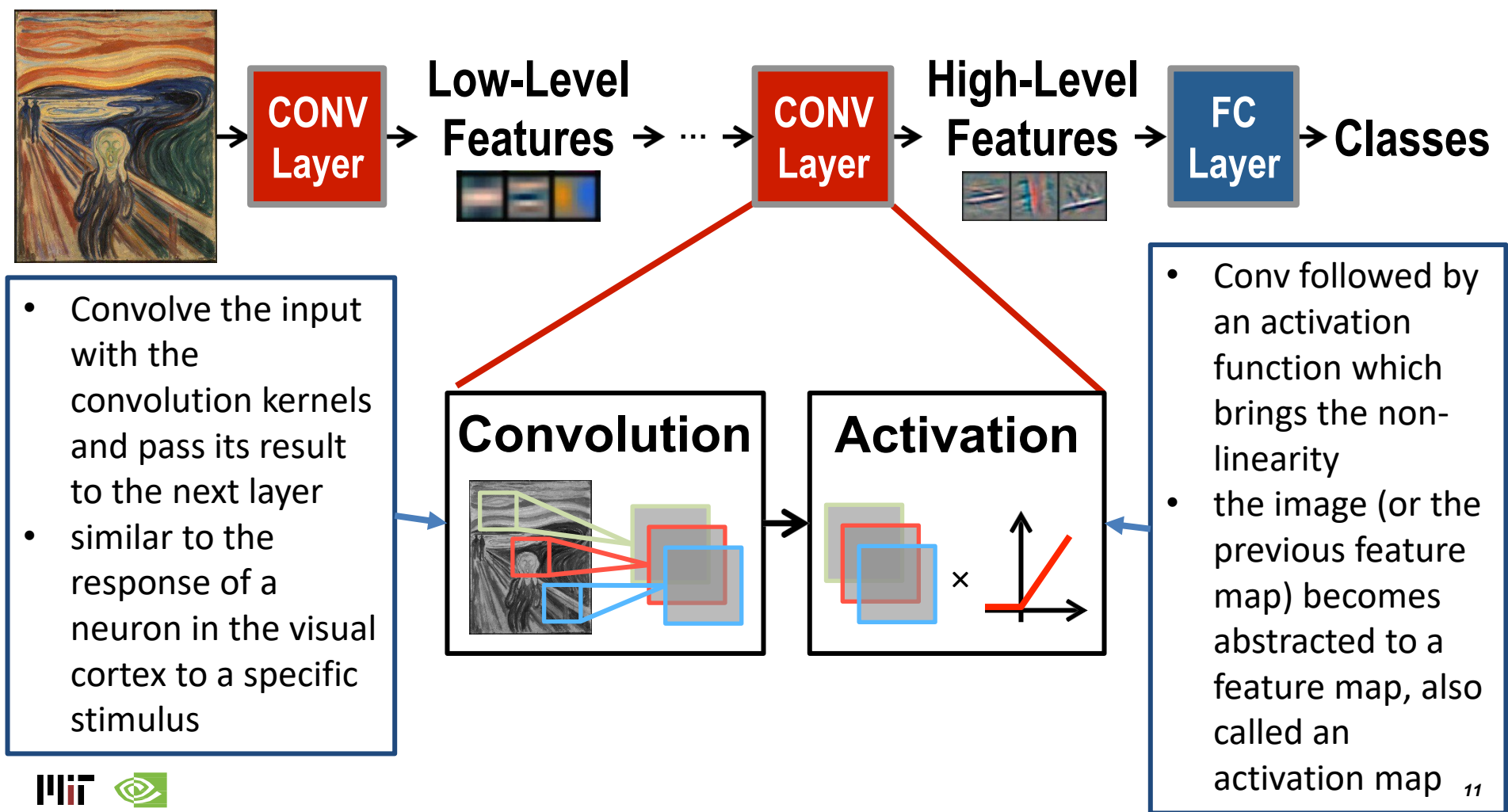
VGG-16 Conv5_3

Deep Convolutional Neural Networks

Modern **Deep** CNN: 5 – 1000 Layers

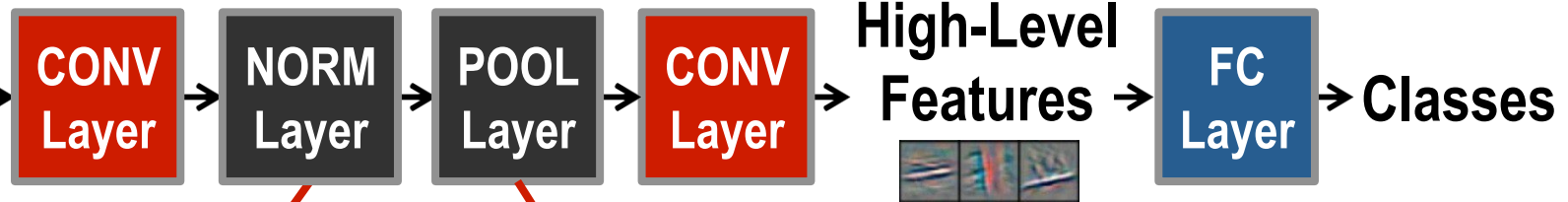
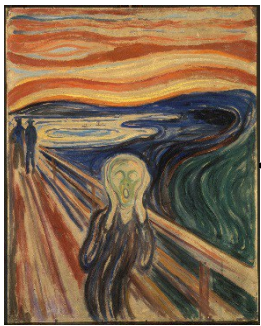


Deep Convolutional Neural Networks

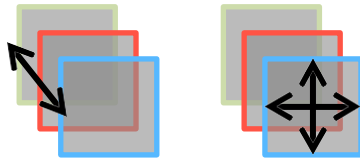


Deep Convolutional Neural Networks

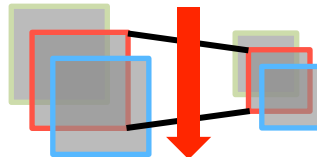
Optional layers in between
CONV and/or FC layers



Normalization

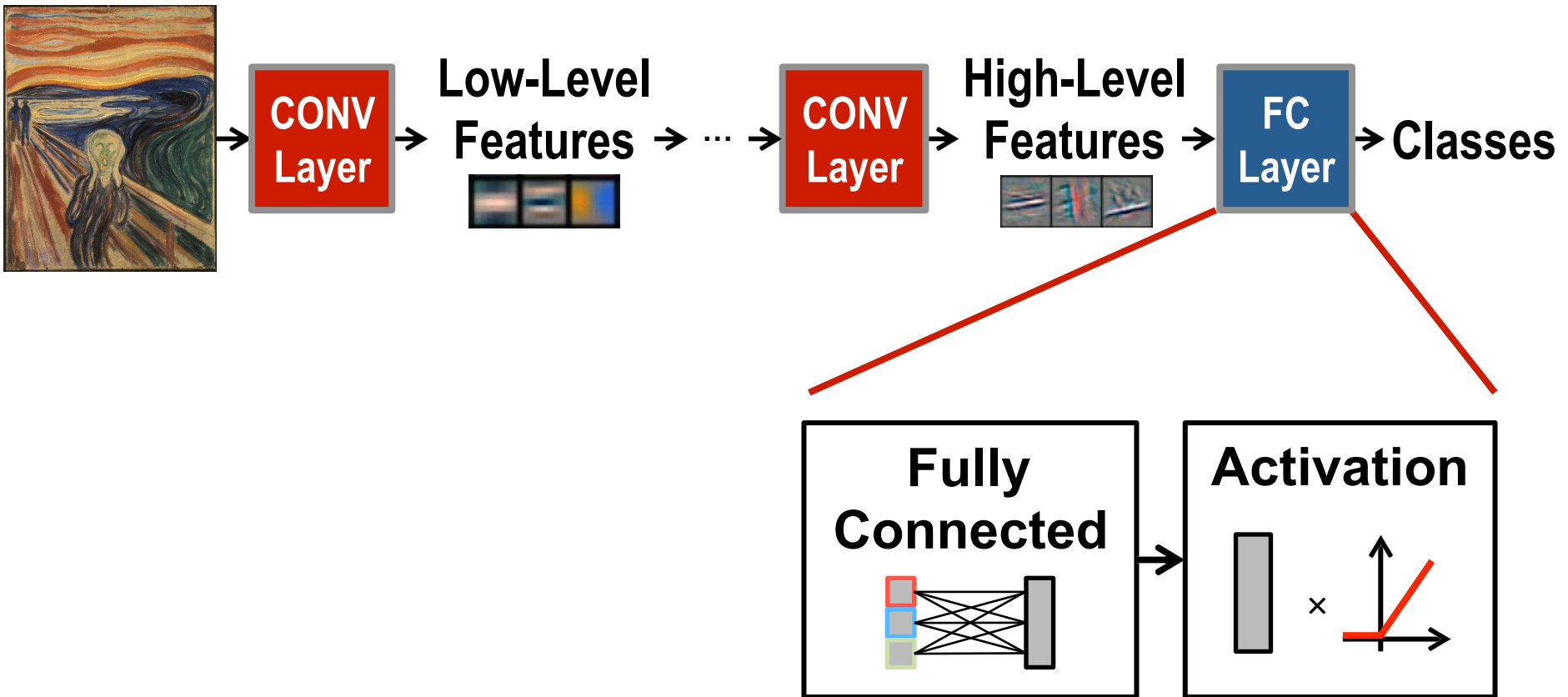


Pooling

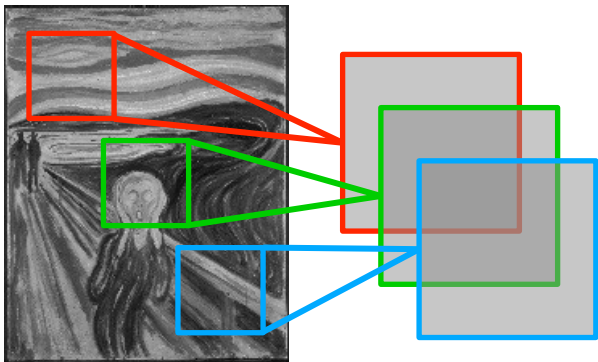
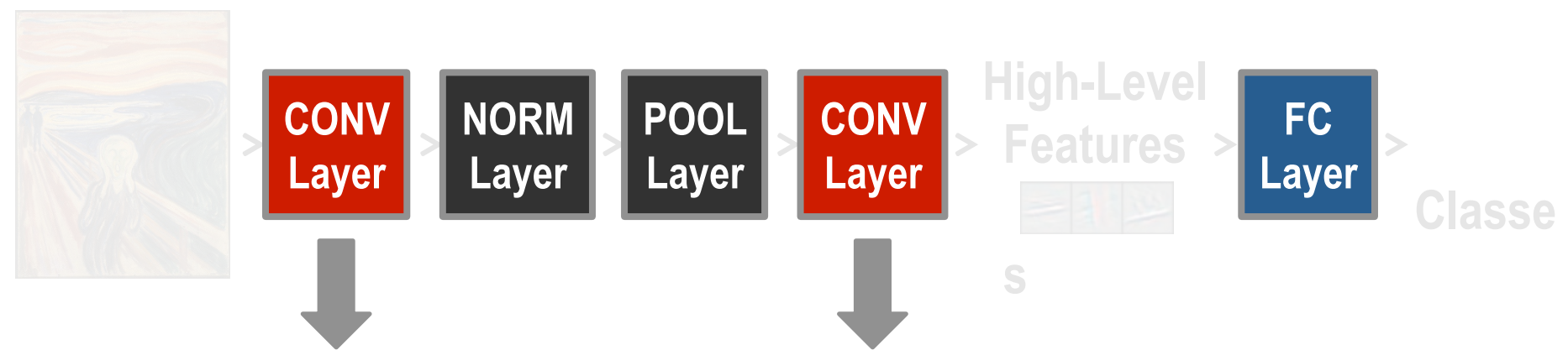


- Pooling layers reduce the dimensions of data by combining the outputs of neuron clusters at one layer into a single neuron in the next layer (down-sampling or sub-sampling) → allow the CNN to be more robust against variations..

Deep Convolutional Neural Networks



Deep Convolutional Neural Networks



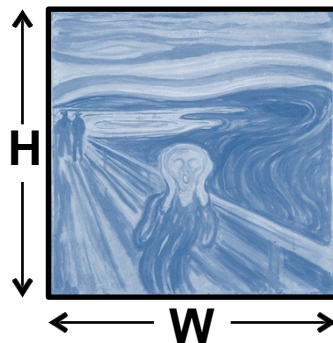
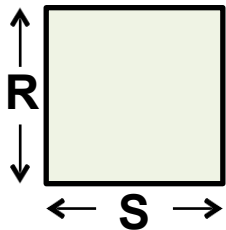
Convolutions account for more than 90% of overall computation, dominating **runtime** and **energy consumption**

CNN Building Blocks

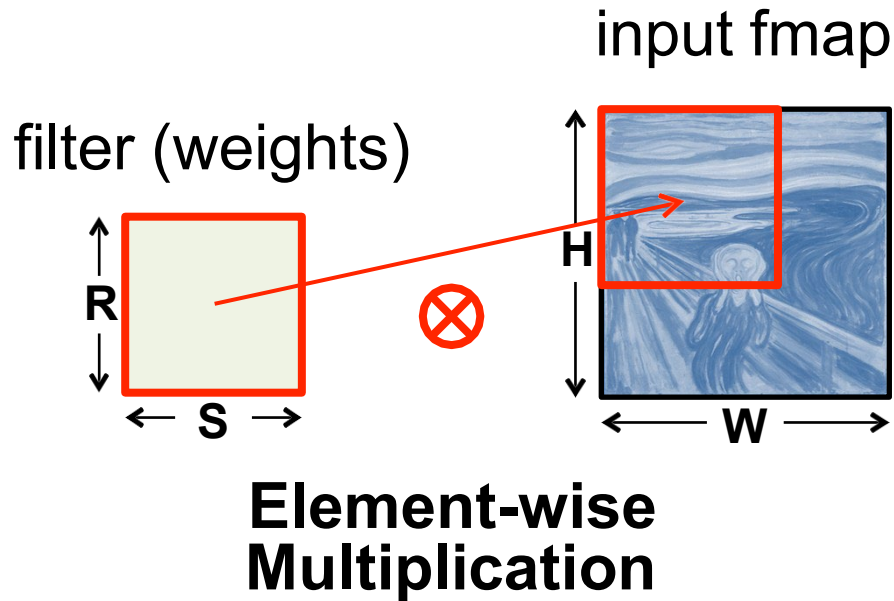
Convolution (CONV) Layer

a plane of input activations
a.k.a. **input feature map (fmap)**

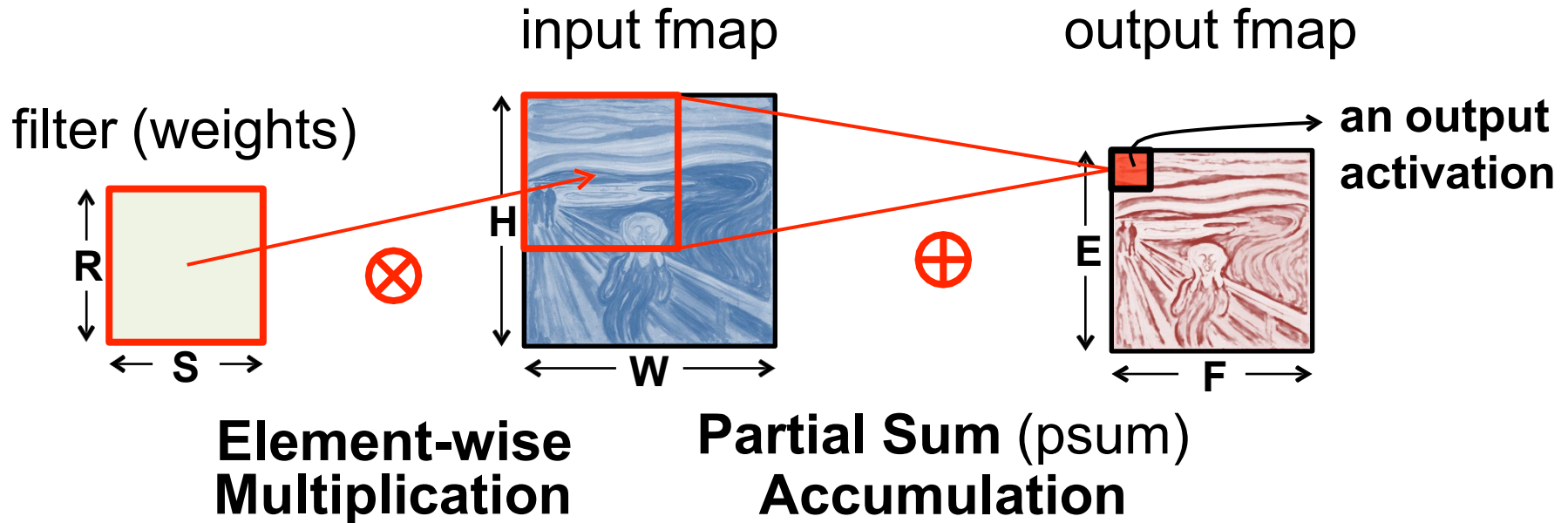
filter (weights)



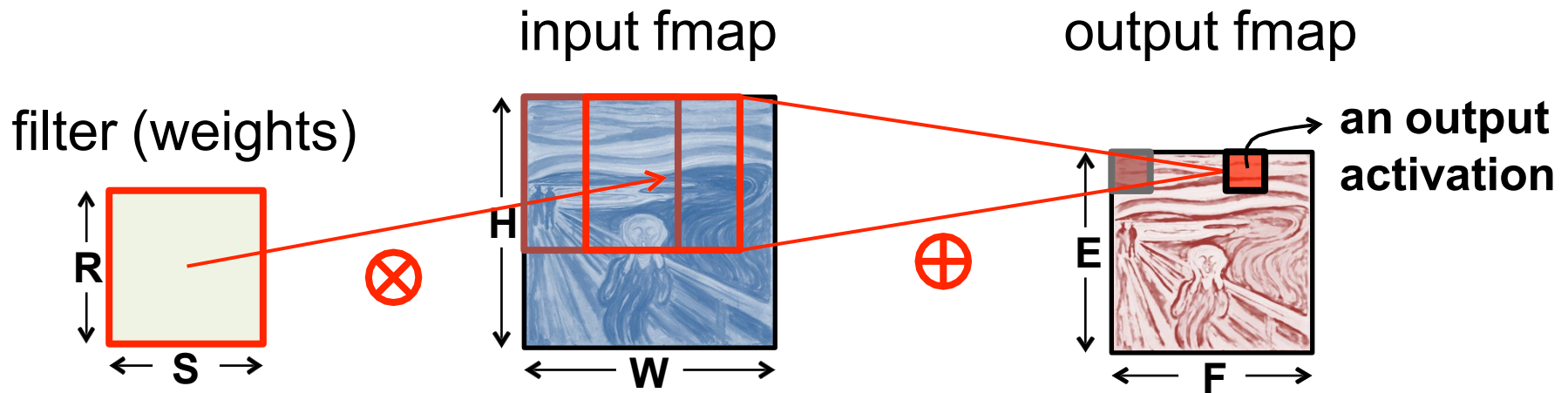
Convolution (CONV) Layer



Convolution (CONV) Layer

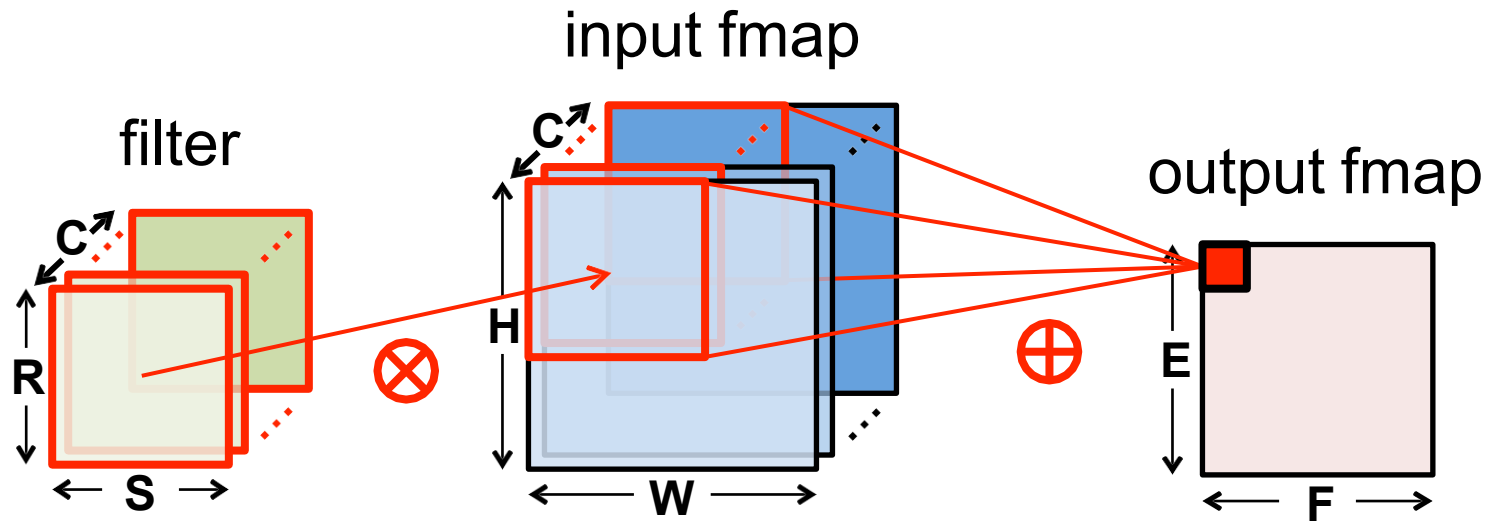


Convolution (CONV) Layer



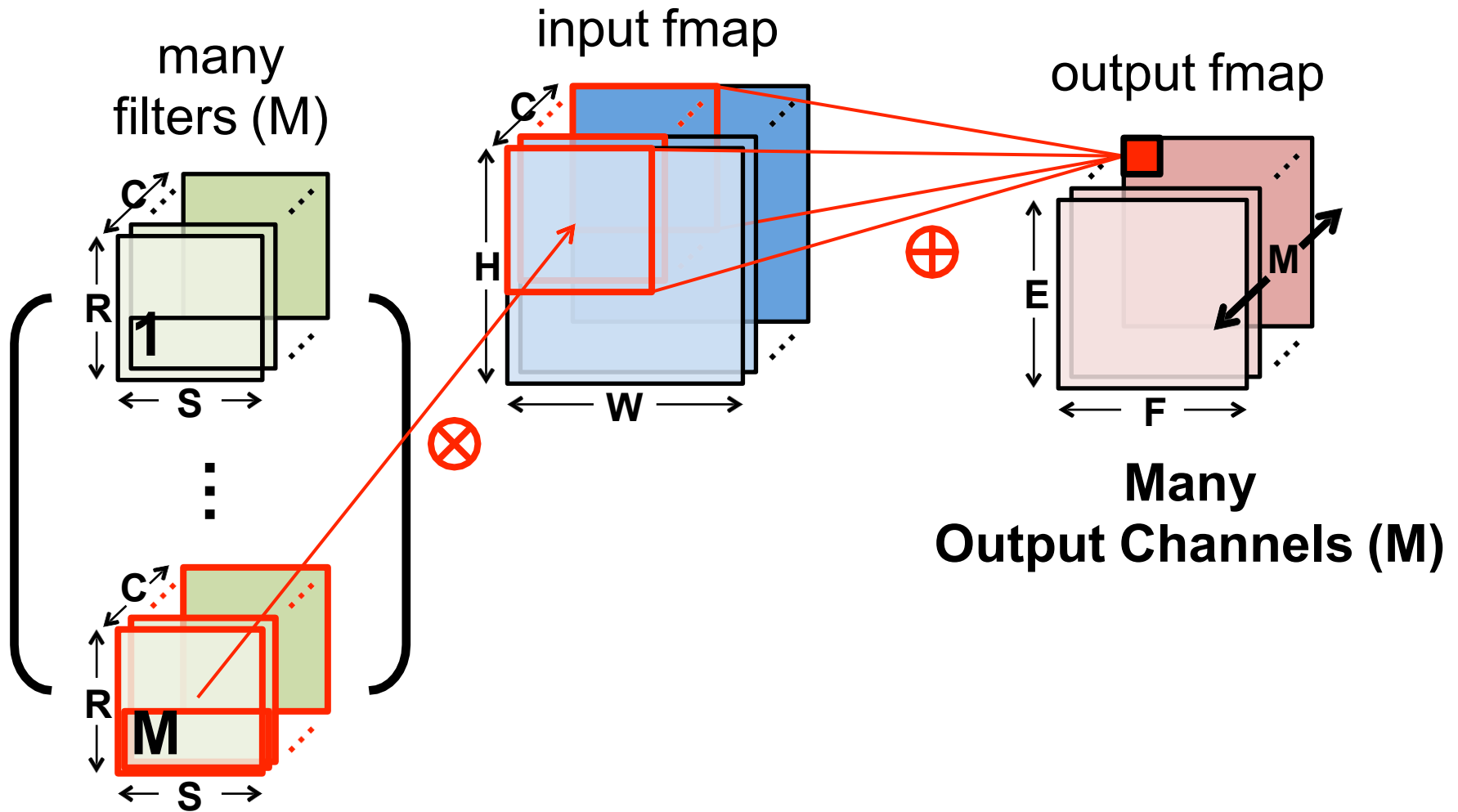
Sliding Window Processing

Convolution (CONV) Layer

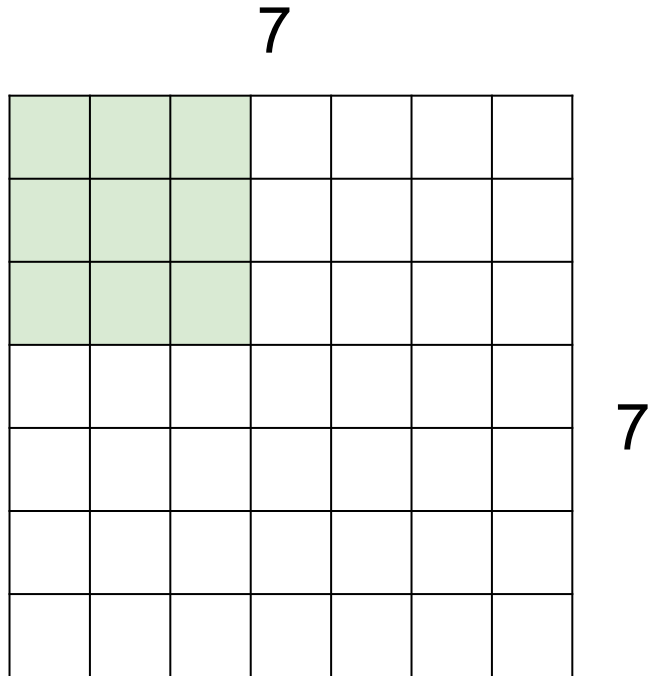


Many Input Channels (C)

Convolution (CONV) Layer



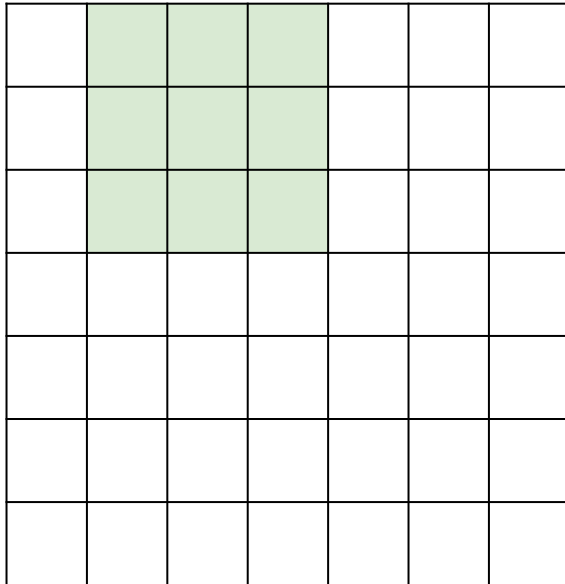
A closer look at spatial dimensions:



7x7 input (spatially)
assume 3x3 filter

A closer look at spatial dimensions:

7

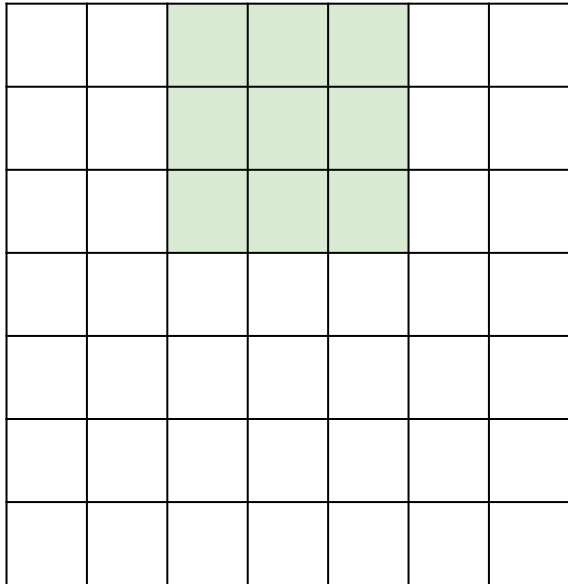


7x7 input (spatially)
assume 3x3 filter

7

A closer look at spatial dimensions:

7

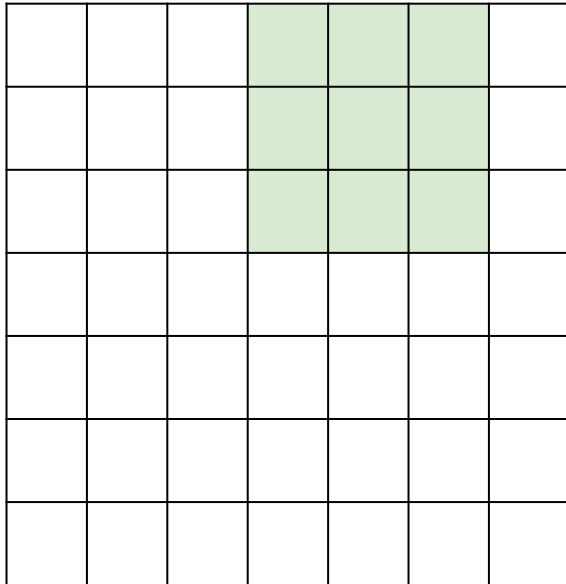


7

7x7 input (spatially)
assume 3x3 filter

A closer look at spatial dimensions:

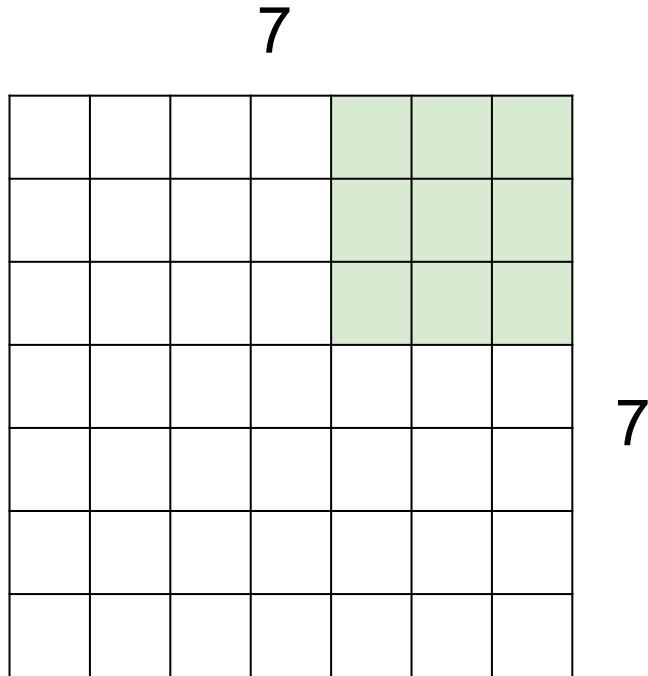
7



7

7x7 input (spatially)
assume 3x3 filter

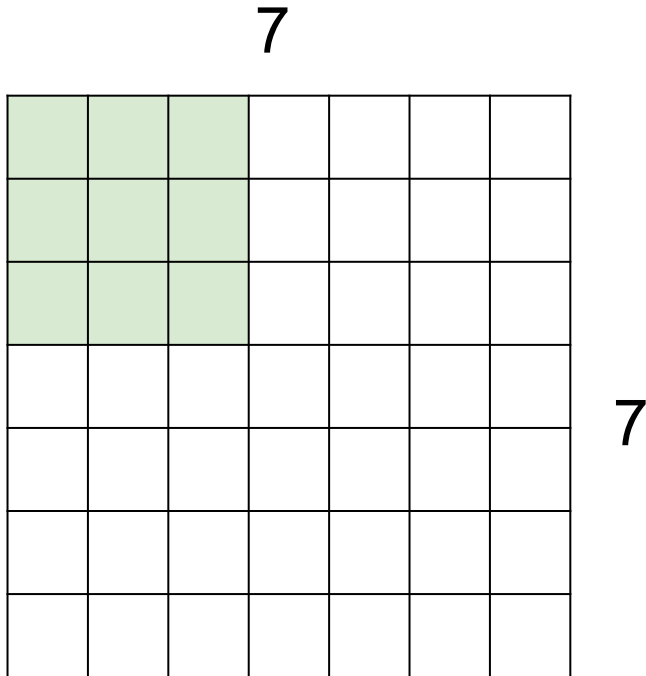
A closer look at spatial dimensions:



7x7 input (spatially)
assume 3x3 filter

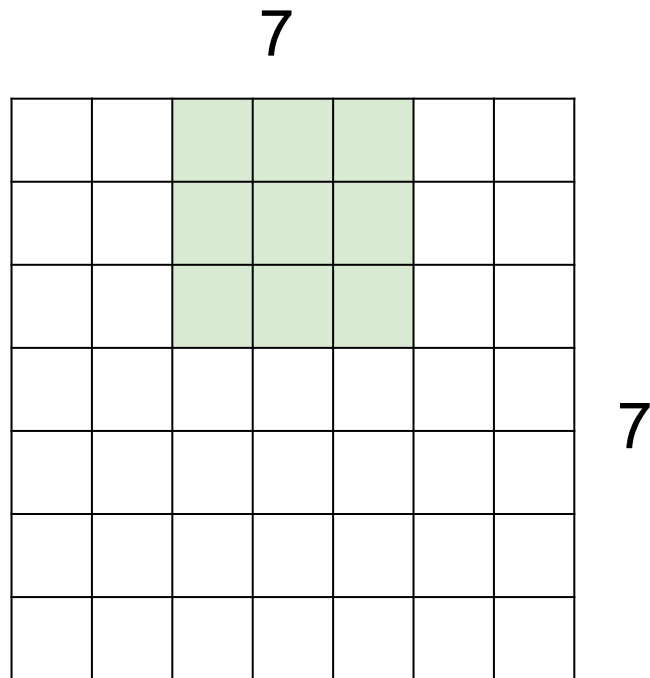
→ **5x5 output**

A closer look at spatial dimensions:



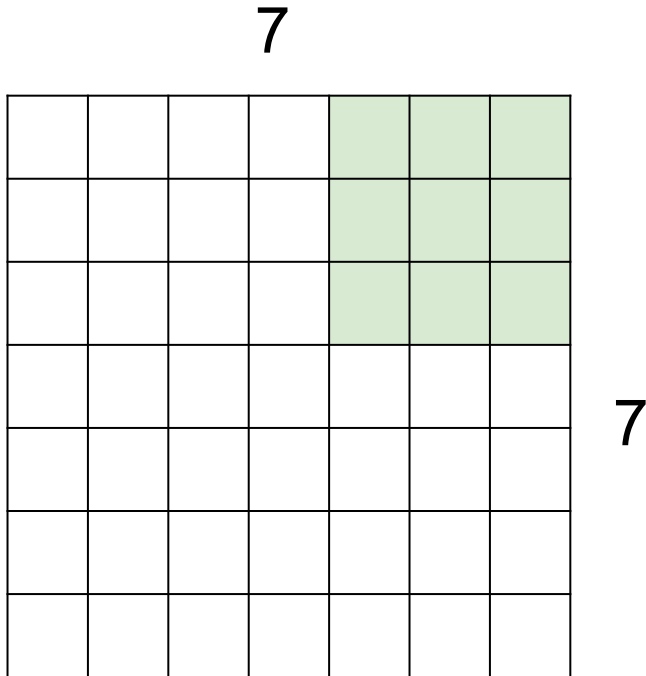
7x7 input (spatially)
assume 3x3 filter
applied **with stride 2**

A closer look at spatial dimensions:



7x7 input (spatially)
assume 3x3 filter
applied **with stride 2**

A closer look at spatial dimensions:



7x7 input (spatially)
assume 3x3 filter
applied **with stride 2**
→ **3x3 output!**

In practice: Common to zero pad the border

0	0	0	0	0	0			
0								
0								
0								
0								

e.g. input 7x7

3x3 filter, applied with **stride 1**

pad with 1 pixel border => what is the output?

(recall:)

$(N - F) / \text{stride} + 1$

In practice: Common to zero pad the border

0	0	0	0	0	0			
0								
0								
0								
0								

e.g. input 7x7

3x3 filter, applied with **stride 1**

pad with 1 pixel border => what is the output?

7x7 output!

(recall:)

$$(N + 2P - F) / \text{stride} + 1$$

In practice: Common to zero pad the border

0	0	0	0	0	0			
0								
0								
0								
0								

e.g. input 7x7

3x3 filter, applied with **stride 1**

pad with 1 pixel border => what is the output?

7x7 output!

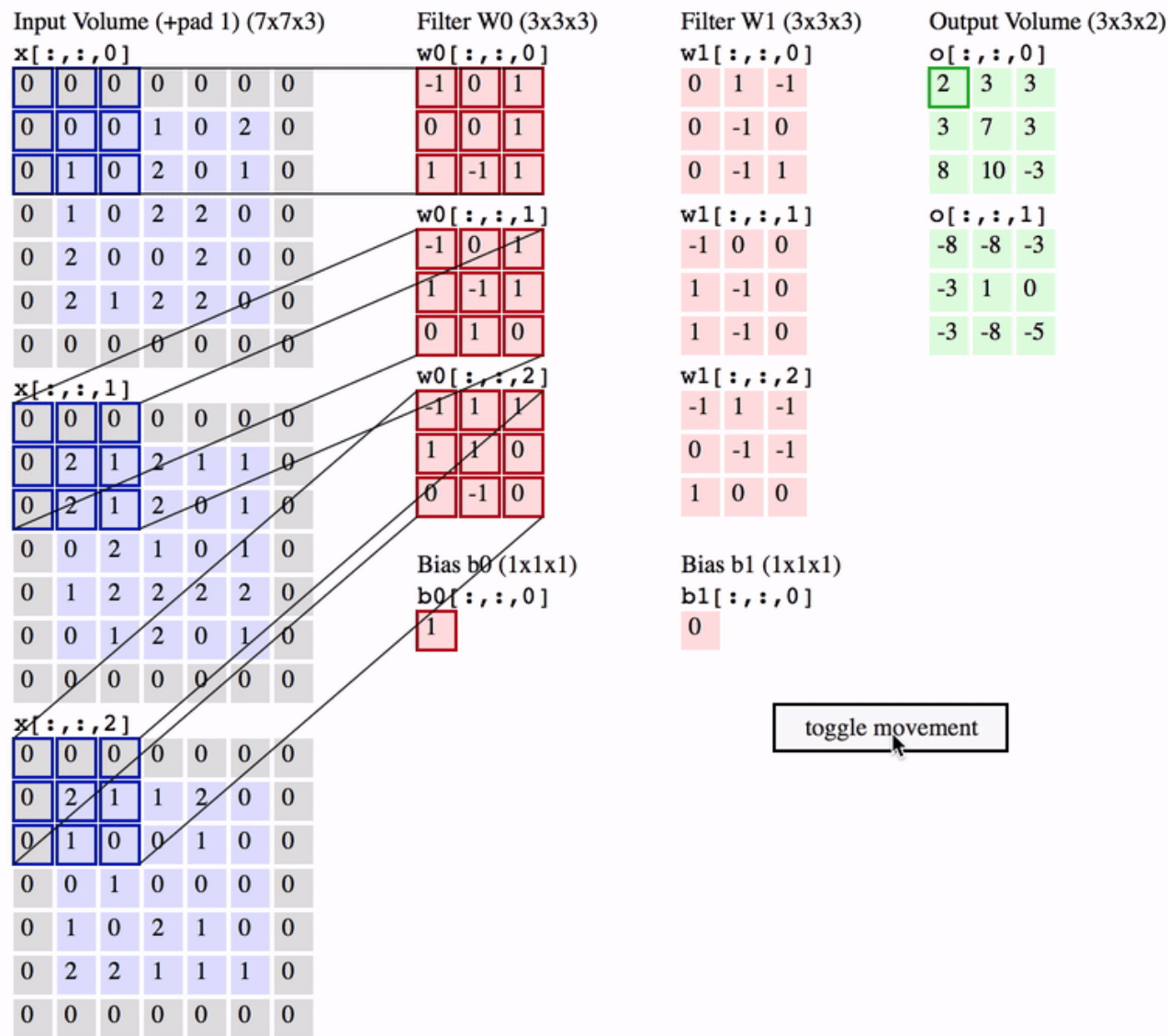
in general, common to see CONV layers with stride 1, filters of size $F \times F$, and zero-padding with $(F-1)/2$. (will preserve size spatially)

e.g. $F = 3 \Rightarrow$ zero pad with 1

$F = 5 \Rightarrow$ zero pad with 2

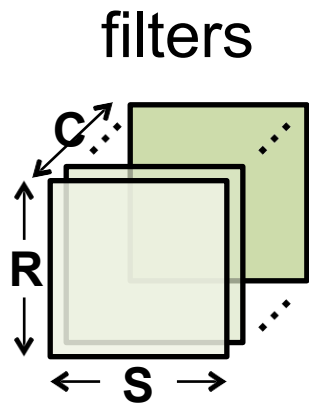
$F = 7 \Rightarrow$ zero pad with 3

CNN computation animated

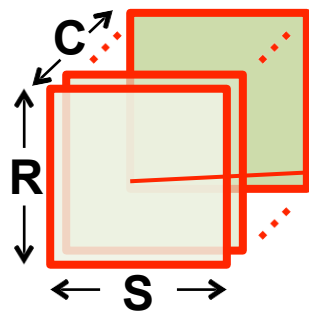


Convolution (CONV) Layer

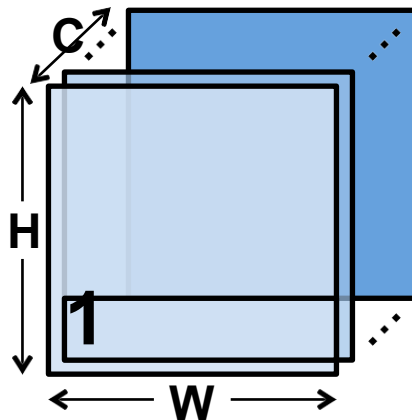
Batching



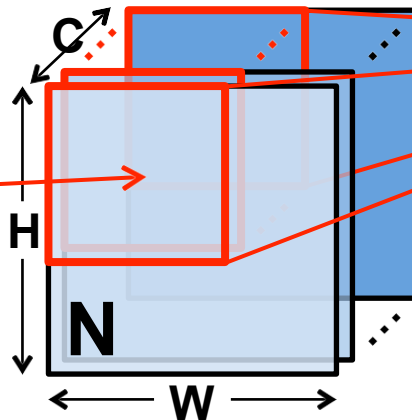
⋮



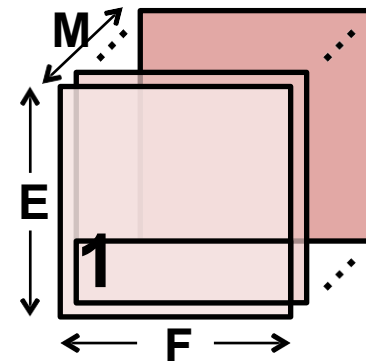
Many
Input fmaps (N)



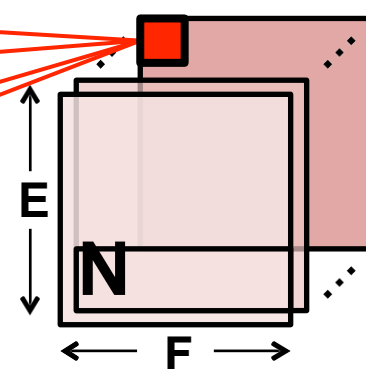
⋮



Many
Output fmaps (N)



⋮



CNN Decoder Ring

- **N** – Number of **input fmaps/output fmaps** (batch size)
- **C** – Number of 2-D **input fmaps /filters** (channels)
- **H** – Height of **input fmap** (activations)
- **W** – Width of **input fmap** (activations)
- **R** – Height of 2-D **filter** (weights)
- **S** – Width of 2-D **filter** (weights)
- **M** – Number of 2-D **output fmaps** (channels)
- **E** – Height of **output fmap** (activations)
- **F** – Width of **output fmap** (activations)

CONV Layer Tensor Computation

Output fmaps (O)

Input fmaps (I)

Biases (B)

Filter weights (W)

$$\underline{O[n][m][x][y]} = \text{Activation}(\underline{B[m]} + \sum_{i=0}^{R-1} \sum_{j=0}^{S-1} \sum_{k=0}^{C-1} \underline{I[n][k][Ux+i][Uy+j]} \times \underline{W[m][k][i][j]}),$$

$$0 \leq n < N, 0 \leq m < M, 0 \leq y < E, 0 \leq x < F,$$

$$E = (H - R + U)/U, F = (W - S + U)/U.$$

Shape Parameter	Description
N	fmap batch size
M	# of filters / # of output fmap channels
C	# of input fmap/filter channels
H/W	input fmap height/width
R/S	filter height/width
E/F	output fmap height/width
U	convolution stride

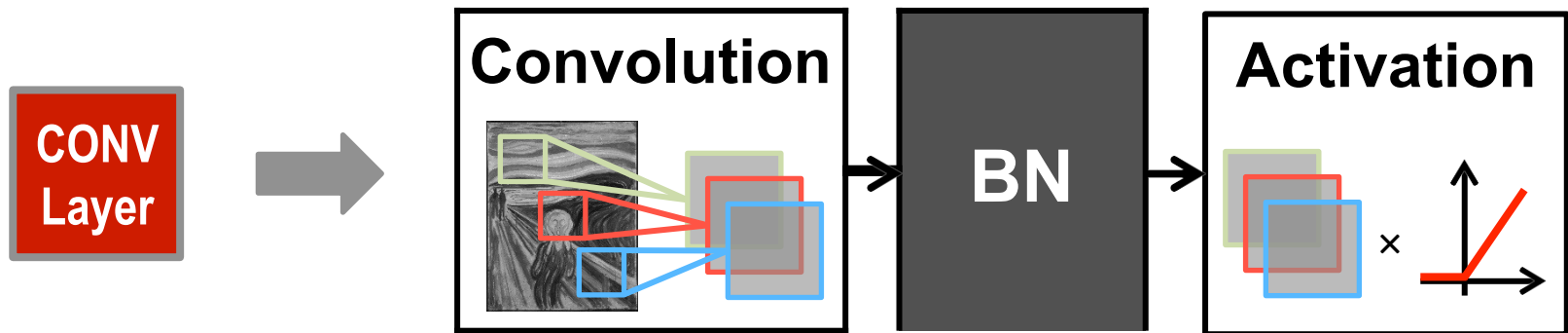
Naïve 7-layer for-loop implementation:

convolve
a window
and apply
activation

Normalization (NORM) Layer

- **Batch Normalization (BN)**

- Normalize activations towards mean=0 and std. dev.=1 based on the statistics of the training dataset
- put **in between** CONV/FC and **Activation function**



Believed to be key to getting high accuracy and faster training on very deep neural networks.

BN Layer Implementation

- The normalized value is further scaled and shifted, the parameters of which are learned from training
- Mean and stdv are computed on the training samples and used during inference.
- Scale factor and shift are learned

The diagram shows the Batch Normalization equation:
$$y = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} \gamma + \beta$$
 with several annotations: a red arrow points from "data mean" to μ ; a red arrow points from "data std. dev." to σ ; a blue arrow points from "learned scale factor" to γ ; a blue arrow points from "learned shift factor" to β ; and a grey arrow points from "small const. to avoid numerical problems" to ϵ .

data mean

learned scale factor

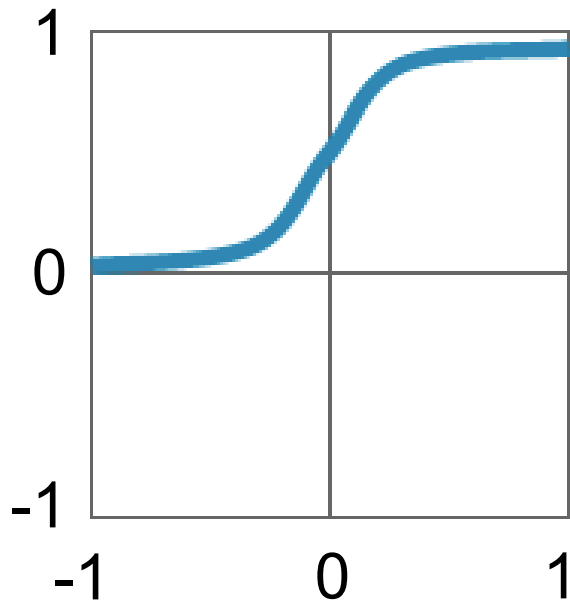
data std. dev.

learned shift factor

small const. to avoid numerical problems

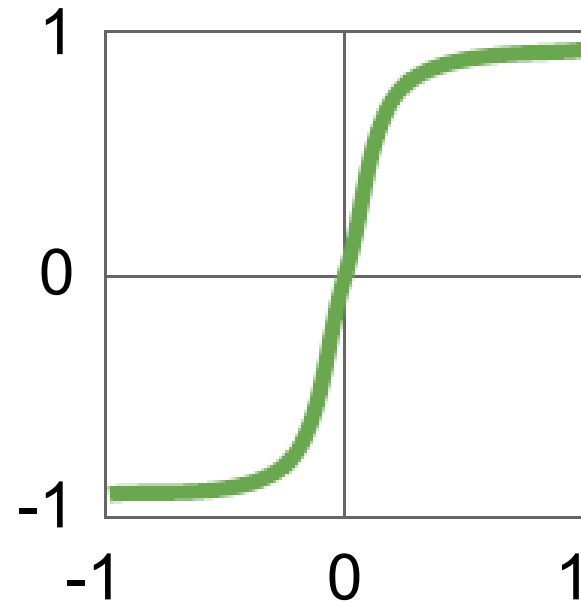
Traditional Activation Functions

Sigmoid



$$y = 1 / (1 + e^{-x})$$

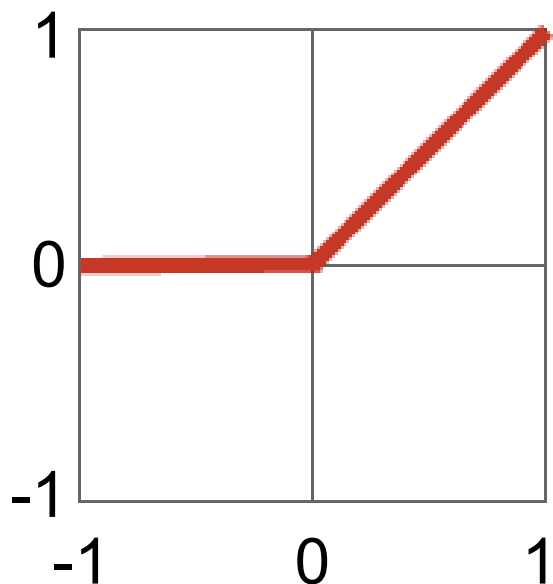
Hyperbolic Tangent



$$y = (e^x - e^{-x}) / (e^x + e^{-x})$$

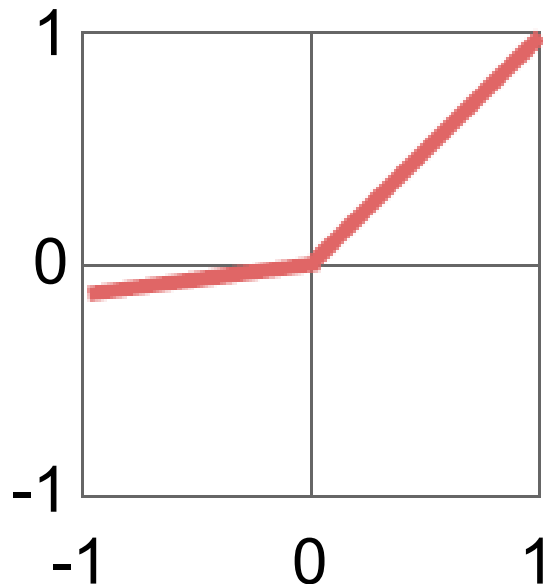
Modern Activation Functions

Rectified Linear Unit
(ReLU)



$$y = \max(0, x)$$

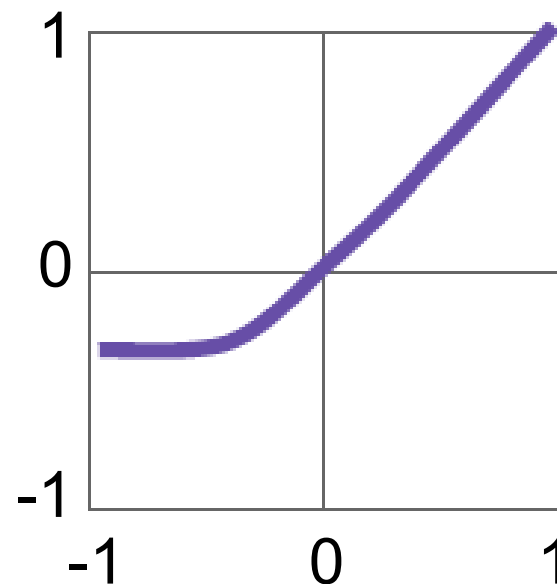
Leaky ReLU



$$y = \max(\alpha x, x)$$

$\alpha = \text{small const. (e.g. 0.1)}$

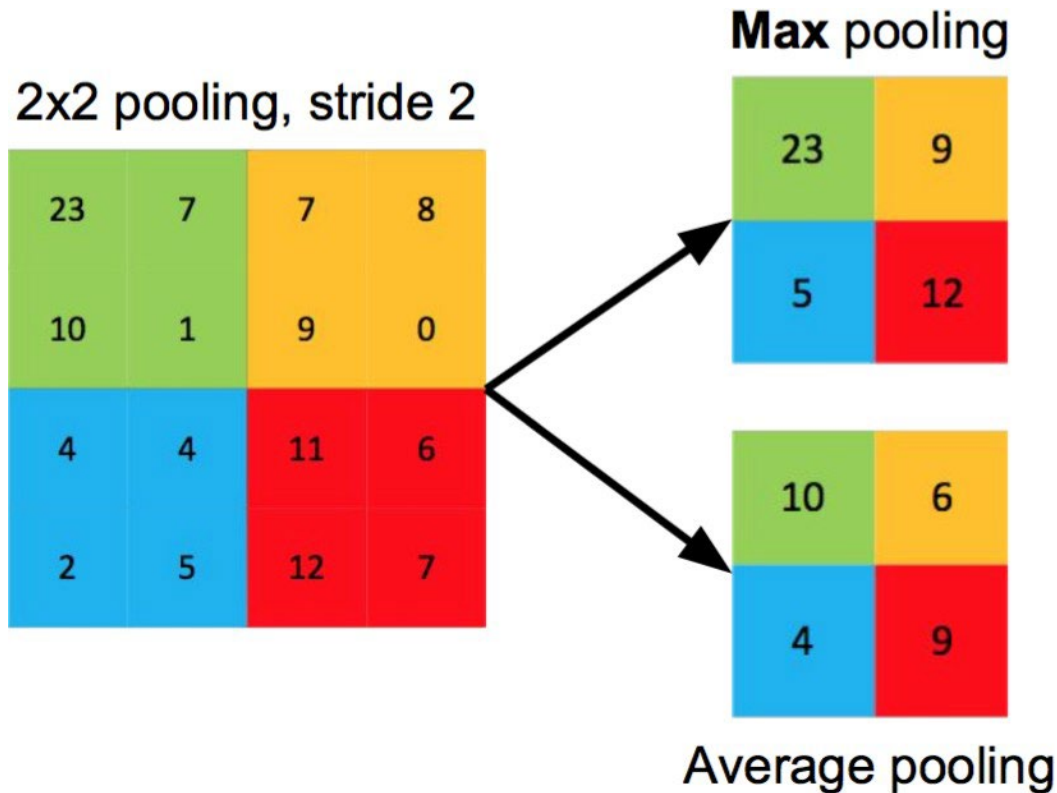
Exponential LU



$$y = \begin{cases} x, & x \geq 0 \\ \alpha(e^x - 1), & x < 0 \end{cases}$$

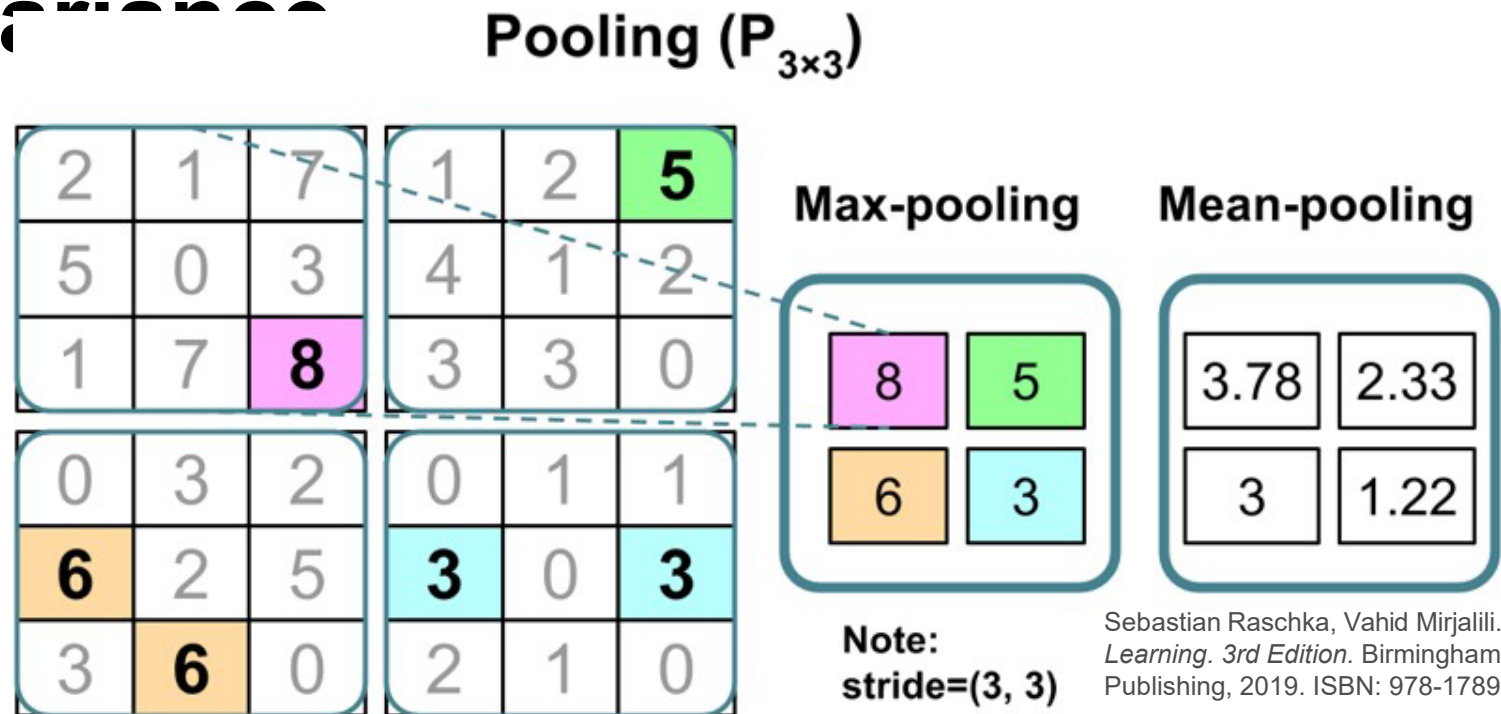
Pooling (POOL) Layer

- Reduce resolution of each channel independently
- Overlapping or non-overlapping → depending on stride



Increases translation-invariance and noise-resilience

Pooling Layers Can Help With Local Invariance



Sebastian Raschka, Vahid Mirjalili. *Python Machine Learning. 3rd Edition*. Birmingham, UK: Packt Publishing, 2019. ISBN: 978-1789955750

Downside: Information is lost.

May not matter for classification, but applications where relative position is important (like face recognition)

In practice for CNNs: some image preprocessing still recommended

POOL Layer Implementation

Naïve 6-layer for-loop max-pooling implementation:

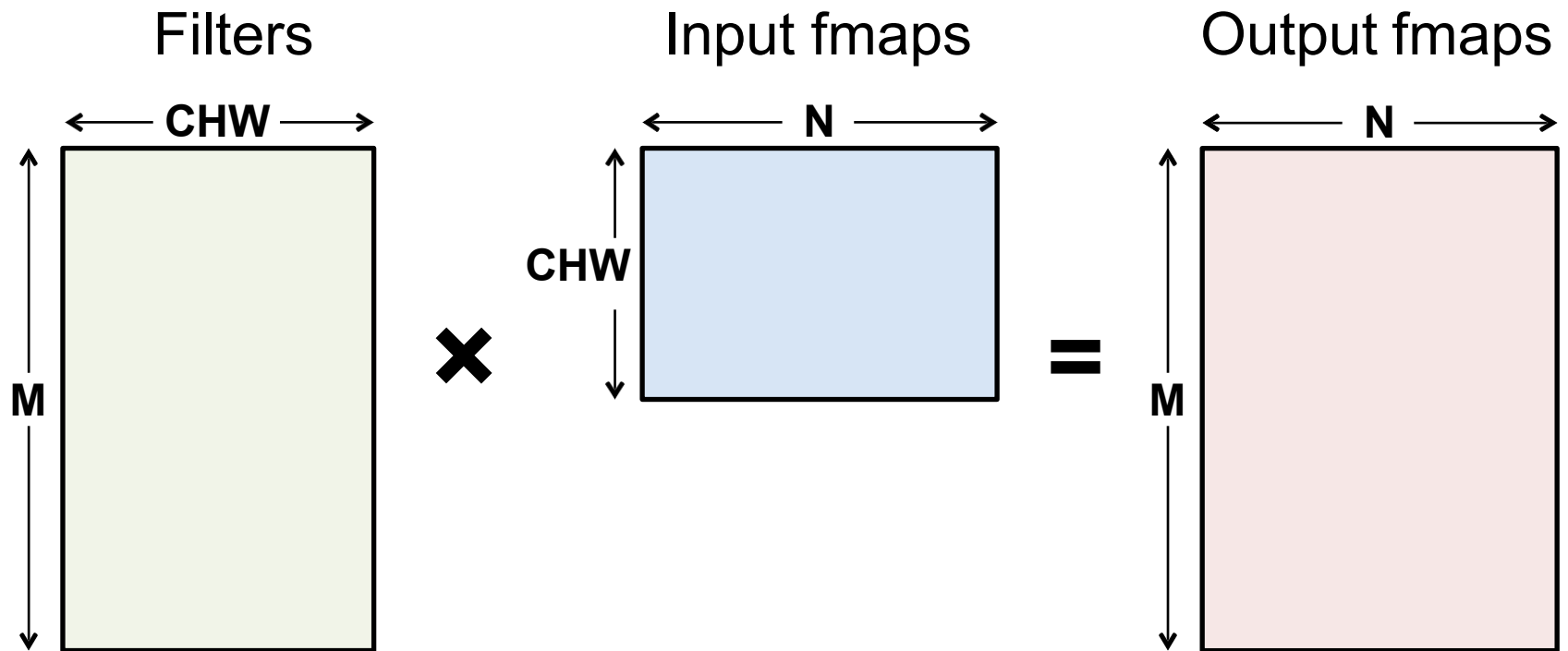
```
for (n=0; n<N; n++) {  
  for (m=0; m<M; m++) {  
    for (x=0; x<F; x++) {  
      for (y=0; y<E; y++) {  
        } } } } } }  
  
        max = -Inf;  
        for (i=0; i<R; i++) {  
          for (j=0; j<S; j++) {  
            if (I[n][m][Ux+i][Uy+j] > max) {  
              max = I[n][m][Ux+i][Uy+j];  
            }  
          }  
        }  
  
        O[n][m][x][y] = max;  
      }  
    }  
  }  
}
```

} for each pooled value

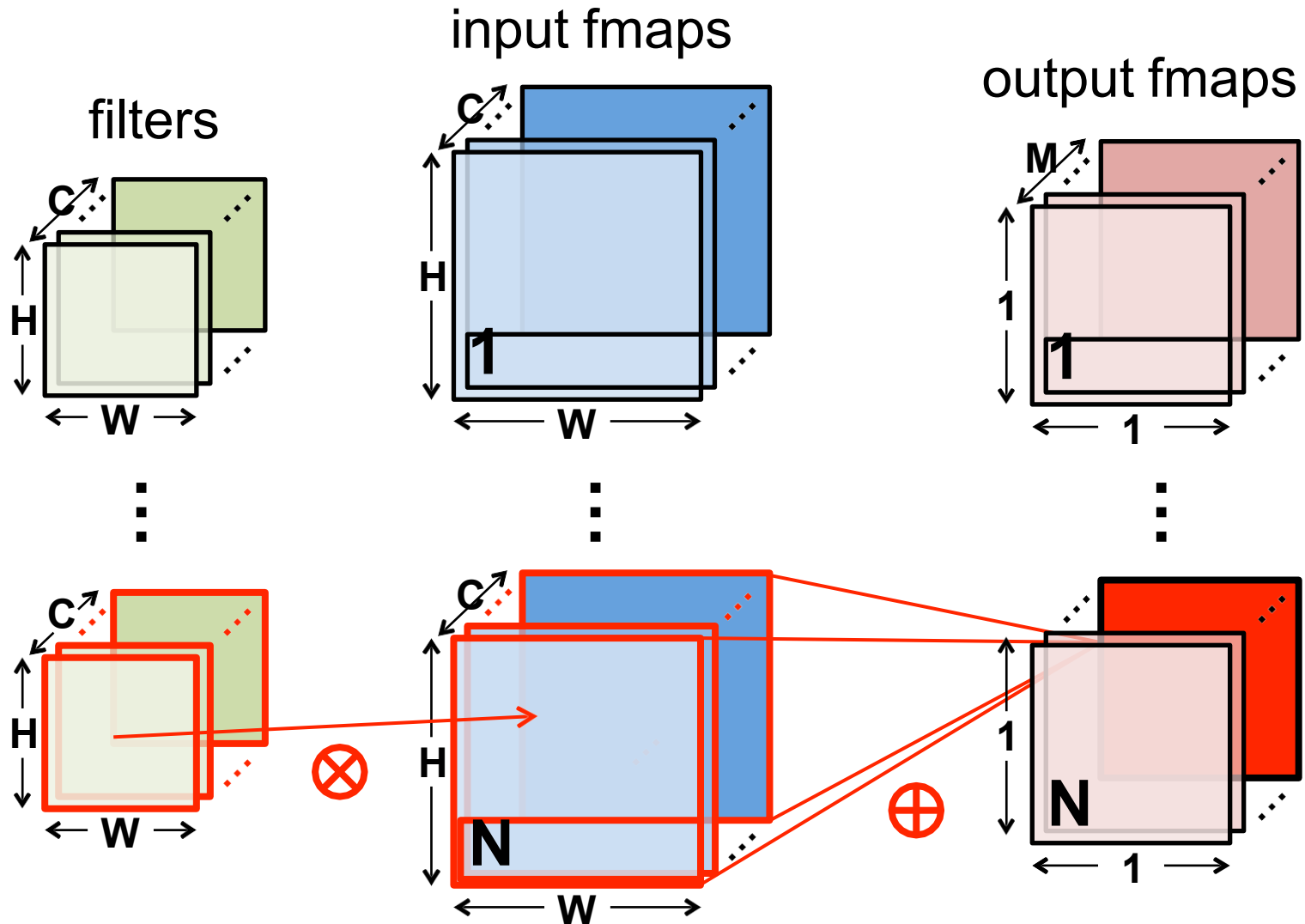
} find the max with in a window

Fully-Connected (FC) Layer

- Height and width of output fmaps are 1 ($E = F = 1$)
- Filters as large as input fmaps ($R = H, S = W$)
- Implementation: **Matrix Multiplication**



FC Layer – from CONV Layer POV



CNN Summary

PROC. OF THE IEEE, NOVEMBER 1998

7

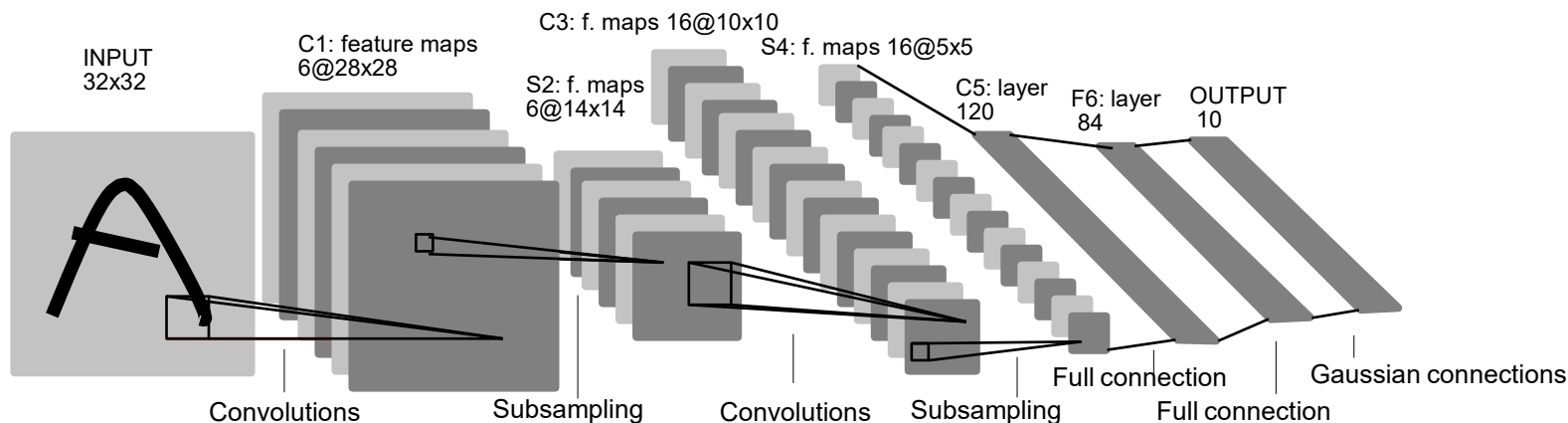


Fig. 2. Architecture of LeNet-5, a Convolutional Neural Network, here for digits recognition. Each plane is a feature map, i.e. a set of units whose weights are constrained to be identical.

Sebastian
Raschka

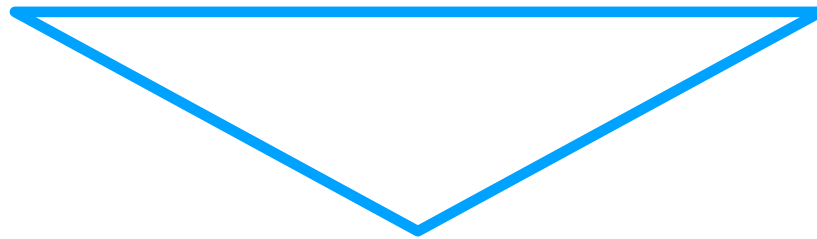
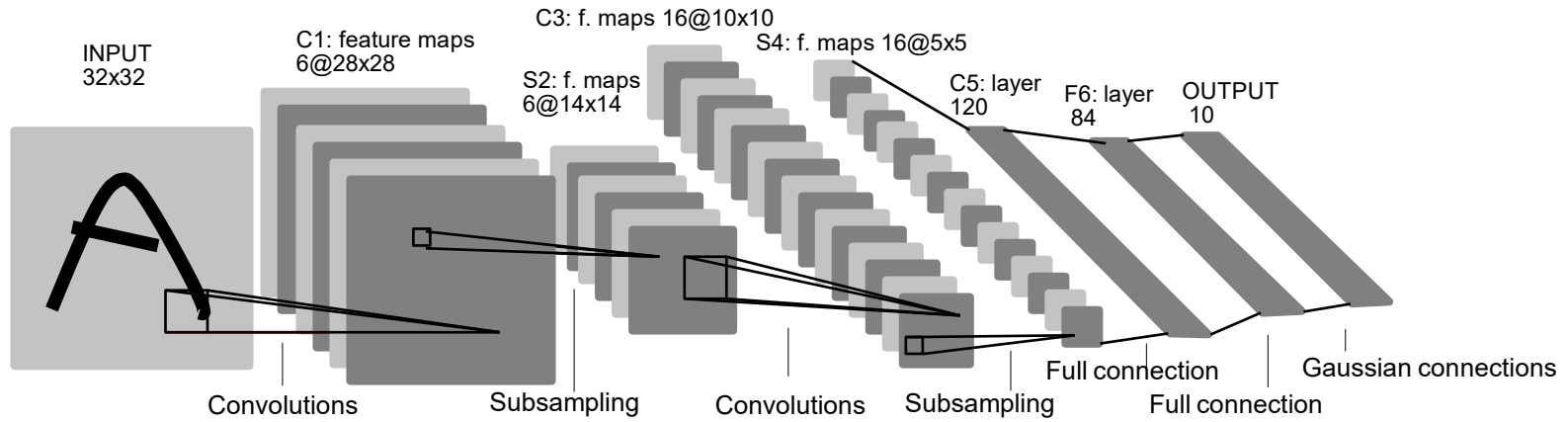
STAT 453: Intro to
Deep Learning

Yann LeCun, Léon Bottou, Yoshua Bengio and Patrick Haffner: [Gradient Based Learning Applied to Document Recognition](#), Proceedings of IEEE, 86(11):2278–2324, 1998.

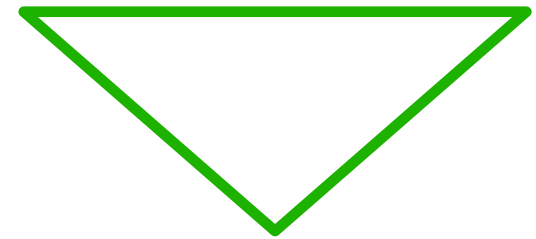
Hidden Layers

PROC. OF THE IEEE, NOVEMBER 1998

7



"Automatic feature extractor"



"Regular classifier"

Hidden Layers

PROC. OF THE IEEE, NOVEMBER 1998

7

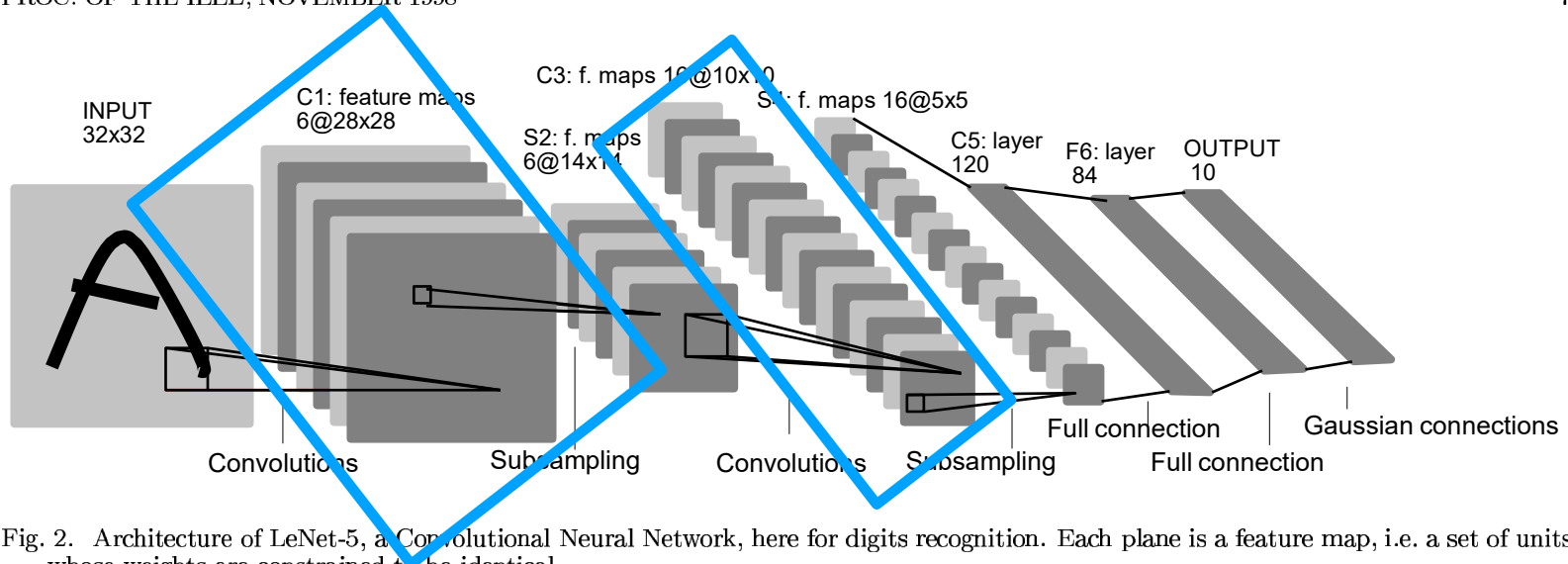


Fig. 2. Architecture of LeNet-5, a Convolutional Neural Network, here for digits recognition. Each plane is a feature map, i.e. a set of units whose weights are constrained to be identical.

Each "bunch" of feature maps represents one hidden layer in the neural network.

Counting the FC layers, this network has 5 layers

CNN key Ideas

PROC. OF THE IEEE, NOVEMBER 1998

7

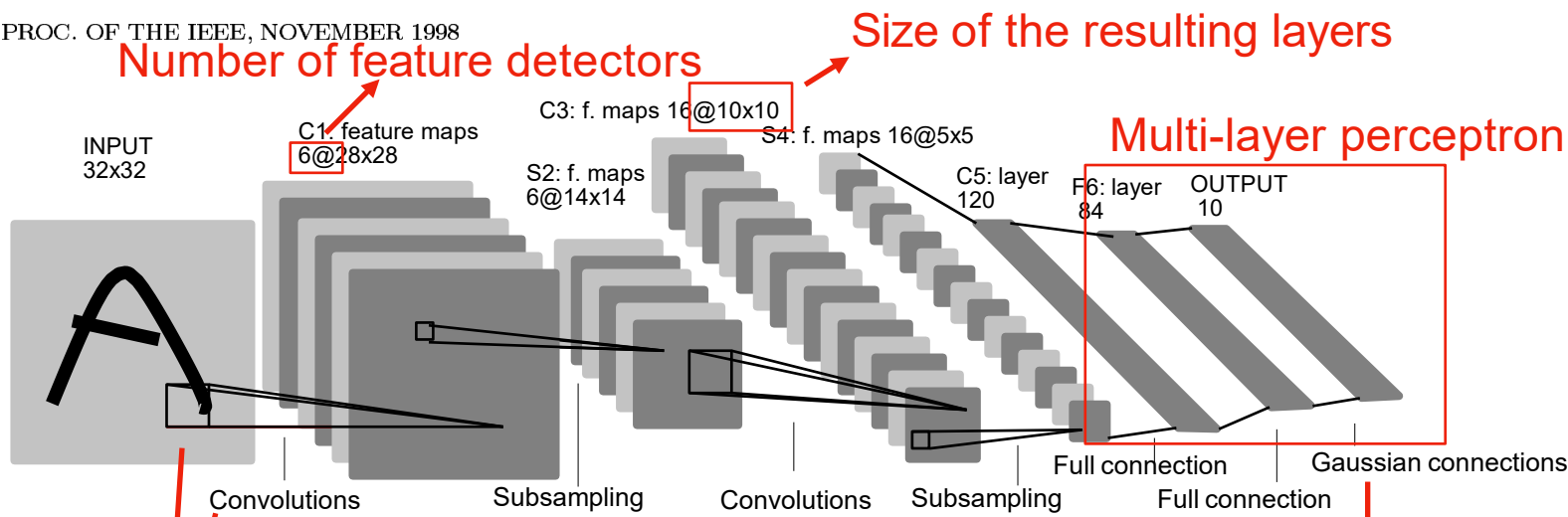


Fig. 2. Architecture of LeNet-5, a Convolutional Neural Network, here for digits recognition. Each plane is a feature map, i.e. a set of units whose weights are constrained to be identical.

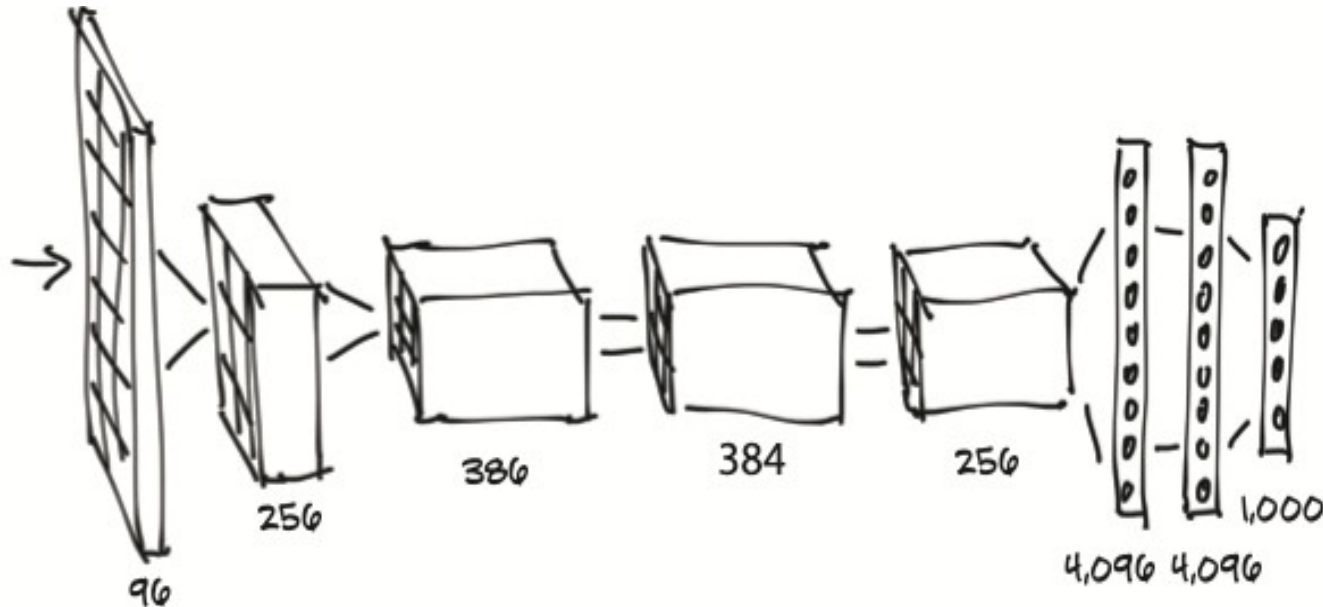
nowadays called "pooling"

"Feature detectors" (weight matrices)
that are being reused ("weight sharing")
=> also called "kernel" or "filter"

basically a fully-connected
layer + MSE loss
(nowadays common to use
fc-layer + softmax
+ cross entropy)

Yann LeCun, Léon Bottou, Yoshua Bengio and Patrick Haffner: [Gradient Based Learning Applied to Document Recognition](#), Proceedings of IEEE, 86(11):2278–2324, 1998.

Main Breakthrough for CNNs: AlexNet & ImageNet



AlexNet achieved 15.4% error on top-5 in
2012 2nd best was not even close: 26.2%
(nowadays ~3% error on ImageNet)

Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). ImageNet classification with deep convolutional neural networks. In *Advances in neural information processing systems* (pp. 1097-1105).

AlexNet

AlexNet

Image (3×224×224)

C × H × W

3×224×224

11×11 Conv, channel 96, stride 4, pad 2

96×55×55

$$\frac{224 + 2 \times 2 - 11}{4} + 1 = 55$$

3×3 MaxPool, stride 2

96×27×27

$$\frac{55 + 0 - 3}{2} + 1 = 27$$

5×5 Conv, channel 256, pad 2, groups 2

256×27×27

$$\frac{27 + 2 \times 2 - 5}{1} + 1 = 27$$

3×3 MaxPool, stride 2

256×13×13

$$\frac{27 + 0 - 3}{2} + 1 = 13$$

3×3 Conv, channel 384, pad 1

384×13×13

$$\frac{13 + 2 \times 1 - 3}{1} + 1 = 13$$

3×3 Conv, channel 384, pad 1, groups 2

384×13×13

$$\frac{13 + 2 \times 1 - 3}{1} + 1 = 13$$

3×3 Conv, channel 256, pad 1, groups 2

256×13×13

$$\frac{13 + 2 \times 1 - 3}{1} + 1 = 13$$

3×3 MaxPool, stride 2

256×6×6

$$\frac{13 + 0 - 3}{2} + 1 = 6$$

Linear, channel 4096

4096

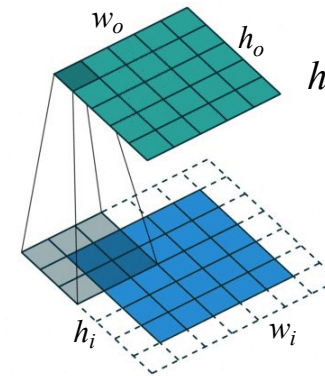
Linear, channel 4096

4096

Linear, channel 1000

1000

Convolution Layer / Pooling Layer

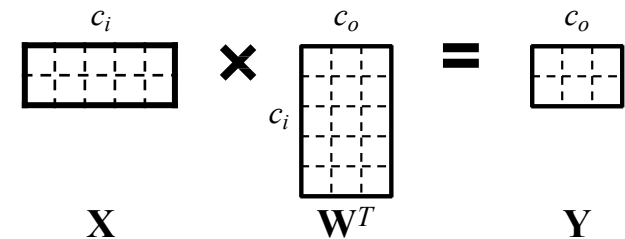


$$h_o = \frac{h_i + 2p - k_h}{s} + 1$$

p is padding

s is stride

Linear Layer



<https://efficientml.ai>